

IT4079:NGÔN NGỮ' và PHƯƠNG PHÁP DỊCH

Phạm Đăng Hải

haipd@soict.hust.edu.vn

Chương 2: Phân tích từ vựng

1. Nhiệm vụ của bộ phân tích từ vựng
2. Biểu thức chính quy
3. Ô tô mát hữu hạn
4. Phân tích từ vựng của ngôn ngữ PL/0

Mục đích & Nhiệm vụ

- Mục đích:
 - Tìm chuỗi dài nhất các ký tự đầu vào, bắt đầu từ ký tự hiện tại tương ứng với một từ tổ và trả về từ tổ này
- Nhiệm vụ
 - Duyệt từng ký tự của văn bản nguồn
 - Loại bỏ các ký tự không cần thiết như dấu cách, chú thích,..
 - Xây dựng từ vựng từ những ký tự đọc được
 - Nhận dạng từ tổ và gửi tới pha tiếp

Nhận biết từ tổ gồm

- Nhận biết các từ khóa, tên do người dùng định nghĩa
- Nhận biết các con số, hằng chuỗi, hằng ký tự
- Nhận biết các ký tự đặc biệt (+,*,...), ký hiệu kép (:=,!=,...)

Từ vựng và Từ tố

- **Từ vựng (Lexeme)**

- Là đơn vị nhỏ nhất trong ngôn ngữ lập trình
 - Được coi là ký hiệu của một bảng chữ của ngôn ngữ
- Được xây dựng từ các ký tự ASCII

- **Từ tố (Token)**

- Là thuật ngữ dùng chỉ các từ vựng có cùng ý nghĩa cú pháp
 - Có thể coi từ vựng là những từ cụ thể trong từ điển: “*hôm nay*”, “*trời*”, “*đẹp*”; còn từ tố là loại từ: “*trạng từ*”, “*danh từ*”, “*tính từ*”, ..

Từ tổ → Ví dụ

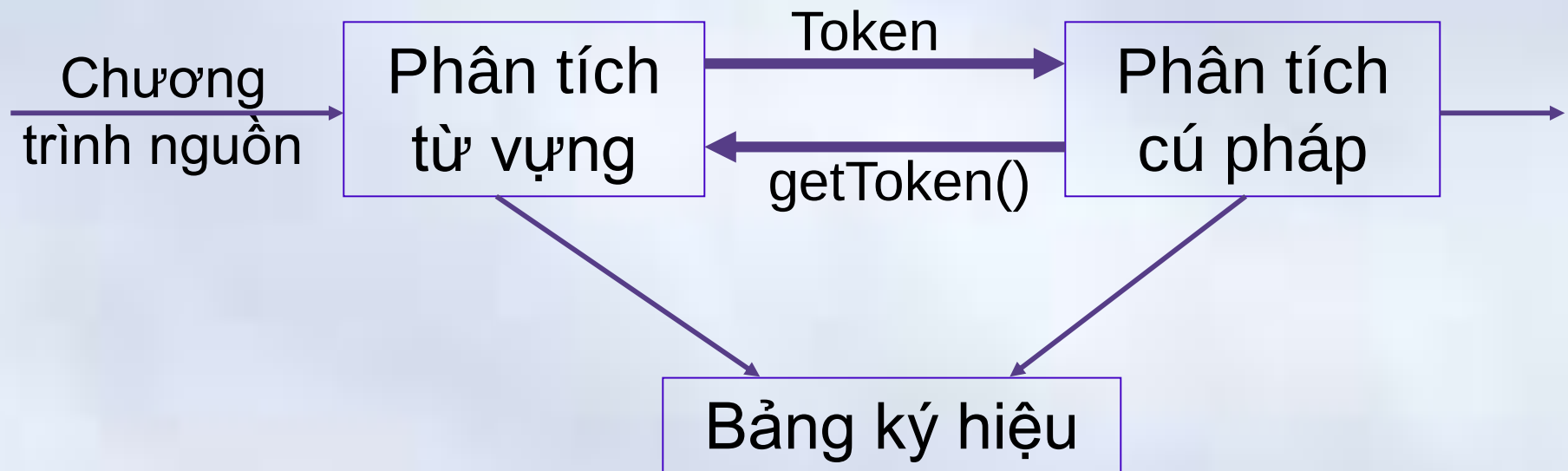
```
pos := start + 10 * size;
```

- “**pos**”, “**start**”, “**size**”, “**+**”, “**10**”, “*****”, “**:=**”, “**;**” là từ vựng
- “**pos**”, “**start**”, “**size**”, → các từ vựng thuộc lớp từ tổ tên (*ident*)
- “**:=**” → từ vựng của từ tổ gán (*assign*)
- “**10**” → từ vựng của từ tổ số nguyên (*number*)
- “**+**” → từ vựng của từ tổ cộng (*plus*)
- “*****” → từ vựng của từ tổ nhân (*times*)
- “**;**” → từ vựng của từ tổ chấm phẩy (*semicolon*)

Từ tổ → Chú ý

- Các từ tổ *Ident, number, plus, assign,...* do người viết trình dịch tự định nghĩa để dễ dàng cho việc mã hóa chương trình. Đây là việc số hóa ký hiệu
- Một từ tổ có thể ứng với tập các từ vựng khác nhau nên cần thêm một số thông tin khác để biết được cụ thể đó là từ vựng nào
 - Các chuỗi “**19**”, “**365**” đều là chuỗi số, có từ tổ “**number**”, nhưng khi sinh mã cần phải biết rõ giá trị là **19** hay **365**
- Bộ phân tích từ vựng không chỉ nhận dạng được các từ tổ mà còn phải biết thuộc tính tương ứng
 - Từ tổ tác động đến bộ phân tích cú pháp
 - Thuộc tính sử dụng trong bộ sinh mã

Thực hiện



- Thực hiện lặp dựa vào yêu cầu từ bộ ptcp
 - Bộ ptcp khi cần một từ tổ sẽ gọi ***getToken()***
 - Nhận được y/cầu, bộ pttv sẽ đọc các ký tự cho tới khi xây dựng xong từ vựng và nhận ra từ tổ hoặc gặp lỗi
- Thường bộ pttv được chia thành 2 phần chính
 - Đọc ký tự
 - Xây dựng từ vựng và nhận dạng từ tổ

Mẫu (Pattern)

- Là luật để mô tả một từ tổ nào đó
 - Cơ sở phân biệt & nhận dạng các từ tổ khác nhau
 - Chuỗi ký tự cùng thỏa mãn một luật $\Rightarrow$ có cùng một từ tổ
 - Từ tổ là tên riêng của một luật mô tả, từ vựng là một trường hợp thỏa mãn luật
- Ví dụ
 - Luật mô tả của từ tổ ***Ident***
 - Bắt đầu là một chữ cái
 - Tiếp theo là tổ hợp chữ cái, chữ số
 - Luật mô tả của từ tổ ***assign***
 - Bắt đầu bởi ký tự “:”, ngay sau đó là ký tự “=“
- Luật được mô tả bởi **biểu thức chính quy**

Chương 2: Phân tích từ vựng

1. Nhiệm vụ của bộ phân tích từ vựng
2. Biểu thức chính quy
3. Ô tô mát hữu hạn
4. Phân tích từ vựng của ngôn ngữ PL/0

Giới thiệu

- **Ngôn ngữ:** Tập hợp của các câu/ xâu (string)
- **Câu:** Dãy hữu hạn của các từ/ký hiệu (symbol)
- **Từ:** Được tạo nên từ một bộ chữ hữu hạn

Ví dụ:

- Ngôn ngữ C là tập các câu lệnh tạo nên các chương trình C hợp lệ
- Bộ chữ cho ngôn ngữ C là tập các chữ cái ASCII
- Một ngôn ngữ có thể là vô hạn, hoặc hữu hạn
- Một ngôn ngữ (*có thể vô hạn*) có thể được mô tả hữu hạn nhờ sử dụng biểu thức chính quy :
 - Mỗi biểu thức đặc trưng cho một tập câu/xâu
 - Chỉ xét xâu có thuộc ngôn ngữ không, chưa xét ý nghĩa của xâu

Biểu thức chính quy (regular expression)

Cho Σ là một bảng chữ của một ngôn ngữ.

- $\emptyset$ là biểu thức chính quy biểu diễn ngôn ngữ $\emptyset$
- ε là biểu thức chính quy biểu diễn ngôn ngữ $\{\varepsilon\}$
- $\forall a \in \Sigma$, a là biểu thức chính quy biểu diễn tập $\{a\}$
- Nếu r và s là các biểu thức chính quy biểu diễn các tập xâu R và S tương ứng thì $(r + s)$, $(r.s)$, (r^*) là các biểu thức chính quy biểu diễn các tập xâu $R \cup S$, RS và R^* tương ứng.

Ngôn ngữ được xác định bởi biểu thức chính quy e , ký hiệu là $L(e)$ là *ngôn ngữ chính quy*

Biểu thức chính quy → Ghi chú

- Biểu thức $(r + s)$ có thể được viết $(r | s)$;
- Biểu thức $(r.s)$ thành (rs)
- Có thể bỏ qua ký hiệu ε
 - $(a |) \Leftrightarrow (a | \varepsilon)$
- Có thể bỏ ngoặc đơn bởi định nghĩa thứ tự ưu tiên
 - Phép đóng Kleene $(*)$ ưu tiên hơn phép ghép $(.)$
 - Phép ghép $(.)$ ưu tiên hơn phép hoặc $(+)$
 - Ví dụ: $(0+(1(0^*))) = (0+(10^*)) = (0+10^*) = 0+10^*$

Biểu thức chính quy $\rightarrow$ Ví dụ

Cho $\Sigma = \{a, b\}$ một bảng chữ.

$$-e_1 = a^* + b^* \Rightarrow L(e_1) = \{\varepsilon, a, aa, aaa, \dots, b, bb, \dots\}$$

$$-e_2 = a^*b^* \Rightarrow L(e_2) = \{\varepsilon, a, b, aa, ab, bb, aaa, \dots\}$$

$$-e_3 = a(a+b)^*$$

$$\Rightarrow L(e_3) = \{a, aa, ab, aaa, aab, aba, abb, \dots\}$$

- Xâu có dạng: bắt đầu là ký hiệu a, tiếp theo là tổ hợp bất kỳ của các ký hiệu a, b
- Nếu a là một chữ cái, b là chữ số

$\Rightarrow L(e_3)$ là ngôn ngữ chứa các tên

$\Rightarrow e_3$ biểu thức chính quy sinh mô tả một tên

Tính chất đại số của btcq

- 2 biểu thức chính quy là tương đương nếu cùng xác định một ngôn ngữ

- Nếu r, s, t là các biểu thức chính quy

$$- r + s = s + r \qquad r + r = r$$

$$- (r + s) + t = r + s + t = r + (s + t)$$

$$- (r.s).t = r.s.t = r.(s.t)$$

$$- r.\varepsilon = \varepsilon.r = r$$

$$- r + \emptyset = \emptyset + r = r$$

$$- r(s+t) = rs + rt \qquad (r+s)t = rt + st$$

$$- r + r^* = r^* ; \quad (r + \varepsilon)^* = r^* ; \quad (r^*)^* = r^*$$

$$- rr^* = r^*r = r +$$

$$- (r+s)^* = (r^*s^*)^*$$

Văn phạm chính quy và Ngôn ngữ chính quy

- **Văn phạm chính quy**

- Văn phạm mà mọi sản xuất có dạng

$$A \rightarrow a|aB \text{ hoặc } A \rightarrow a|Ba$$

- Dùng diễn tả từ vựng của NNLT

$$\langle \text{Tên} \rangle \rightarrow \langle \text{Chữ cái} \rangle | \langle \text{Tên} \rangle \langle \text{Chữ cái} \rangle | \langle \text{Tên} \rangle \langle \text{Chữ số} \rangle$$

$$\langle \text{Tên} \rangle \rightarrow "a" | "b" | "c" | \dots | "z" | "A" | "B" | \dots | "Z"$$

$$\langle \text{Chữ số} \rangle \rightarrow "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"$$

- Văn phạm chính quy sinh ra ngôn ngữ chính quy

- **Ngôn ngữ chính quy**

- Được biểu diễn (*mô tả*) bởi biểu thức chính quy
- Đoán nhận bởi các Otomat hữu hạn

Ngôn ngữ chính quy $\rightarrow$ Văn phạm chính quy

r là biểu thức chính qui cần chuyển

1. Thêm ký hiệu khởi đầu **S** và tạo sản xuất **$S \rightarrow r$**
2. Loại bỏ khỏi văn phạm các siêu ký hiệu của r
 - $\forall$ SX dạng **$A \rightarrow r_1.r_2$** : Thêm ký hiệu không kết thúc B và thay thành 2 SX:

$$A \rightarrow r_1 B \quad \& \quad B \rightarrow r_2$$

- $\forall$ SX dạng **$A \rightarrow r_1 + r_2$** : Thay bởi **$A \rightarrow r_1 | r_2$**
- $\forall$ SX dạng **$A \rightarrow r_1^* r_2$** : Thêm ký hiệu không kết thúc **B** và thay thành 4 sản xuất

$$A \rightarrow r_1 B; \quad A \rightarrow r_2$$

$$B \rightarrow r_1 B; \quad B \rightarrow r_2$$

Ví dụ : Chuyển đổi biểu thức $e = a(a+b)^*$

1. Thêm S và sản xuất $S \rightarrow a(a+b)^*$
2. Loại bỏ các ký hiệu không thuộc bộ chữ

– Xét $r = a(a+b)^* // r_1 = a; r_2 = (a+b)^*$

Thêm A và các SX $S \rightarrow aA$ & $A \rightarrow (a+b)^*$

– Xét $A \rightarrow (a+b)^* // r_1 = a+b; r_2 = \varepsilon$

Thêm B và các SX

$A \rightarrow (a+b)B$ & $A \rightarrow \varepsilon$ & $B \rightarrow (a+b)$ & $B \rightarrow \varepsilon$

– Áp dụng luật phân phối phải

$A \rightarrow aB+bB$ & $A \rightarrow \varepsilon$ & $B \rightarrow aB+bB$ & $B \rightarrow \varepsilon$

$A \rightarrow aB|bB| \varepsilon$ và $B \rightarrow aB|bB| \varepsilon$

Ví dụ : Chuyển đổi biểu thức $e = a(a+b)^*$ (tiếp)

- Loại bỏ ký hiệu ε bởi tạo ra xâu mới

$$B \rightarrow aB|bB| \varepsilon \text{ thành } B \rightarrow aB|bB| a | b$$

$$A \rightarrow aB|bB| \varepsilon \text{ thành } A \rightarrow aB|bB| a | b$$

Vai trò của A và B là tương đương. Thay ký hiệu B bằng A và loại bỏ B

Kết quả: Văn phạm cuối

$$S \rightarrow a | aA$$

$$A \rightarrow aA | bA | a | b$$

Bài tập

- Tìm văn phạm chính quy tương với các biểu thức chính quy sau
 - $1(0+1)^*1$
 - $1+01^*$

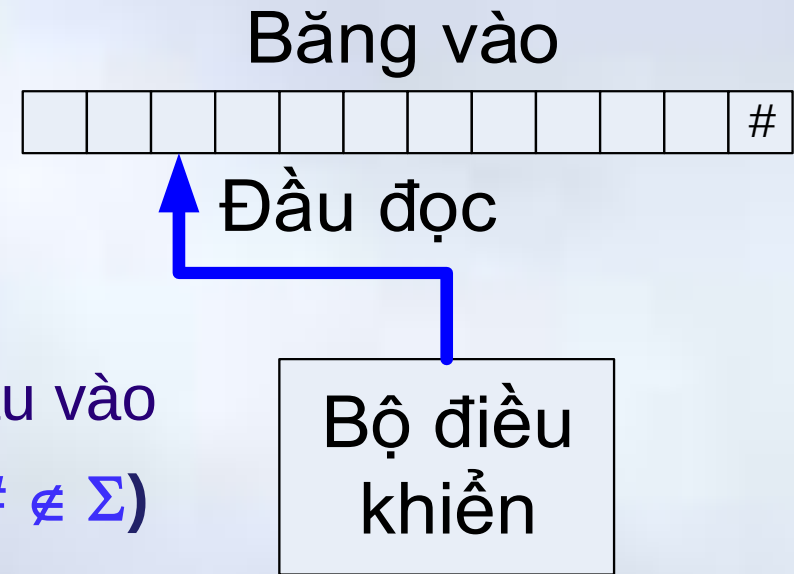
Chương 2: Phân tích từ vựng

1. Nhiệm vụ của bộ phân tích từ vựng
2. Biểu thức chính quy
3. Ô tô mát hữu hạn
4. Phân tích từ vựng của ngôn ngữ PL/0

Mô tả

Là một hệ thống gồm

- Băng vào
 - Là dãy các ô nhớ
 - Mỗi ô chứa một ký hiệu của xâu vào
 - Xâu vào thuộc bộ chữ vào Σ ($\# \notin \Sigma$)
- Đầu đọc
 - Mỗi thời điểm đọc được một ô trên băng vào
 - Chỉ có thể dịch chuyển sang phải
- Bộ điều khiển
 - Gồm một tập các trạng thái Q
 - Có một trạng thái đầu $q_0 \in Q$ và một tập trạng thái kết thúc $F \subseteq Q$
 - Hoạt động theo hàm dịch chuyển $\delta: (Q \times \Sigma) \rightarrow Q$



Hoạt động

Ô-tô-mát hữu hạn hoạt động theo từng bước

- Tại mỗi thời điểm, tùy theo
 - Trạng thái hiện tại của bộ điều khiển
 - Ký hiệu đọc được trên băng vào

Ô-tô-mát sẽ

- Dịch chuyển đầu đọc sang phải một ô
- Chuyển sang trạng thái mới theo hàm dịch chuyển

Đoán nhận chuỗi $\omega \in \Sigma^*$

- Đặt chuỗi cần đoán nhận ω lên băng vào
 - Mỗi ký hiệu một ô
 - Ký hiệu #, đặt ở cuối dùng đánh dấu kết thúc chuỗi vào
- Ô-tô-mát khởi đầu từ trạng thái q_0
- Đầu đọc trở vào ký hiệu trái nhất của ω (*Là ô đầu tiên của băng vào*)

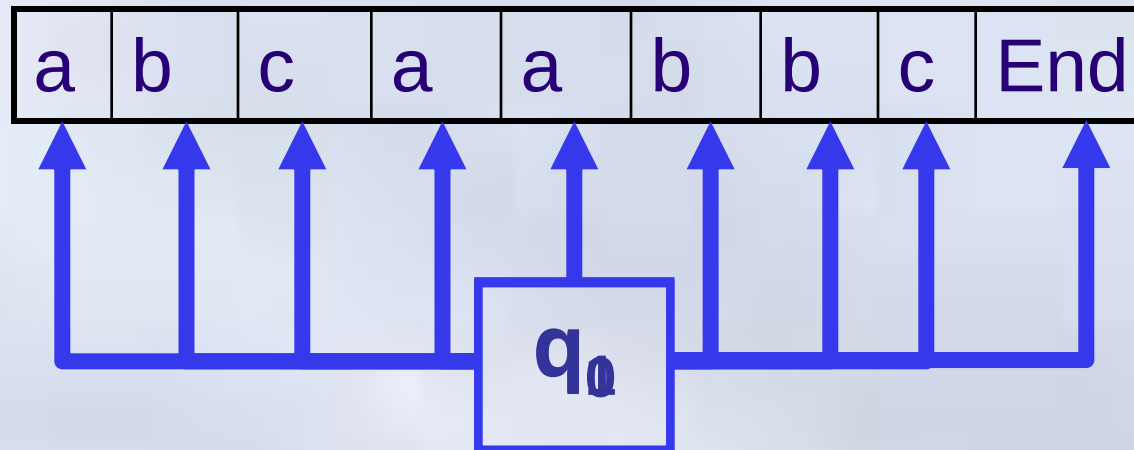
Ô-tô-mát dịch chuyển một số hữu hạn các bước cho tới khi đọc xong chuỗi ω (đầu đọc trở tới #)

- Ô-tô-mát ở trạng thái kết thúc, chuỗi được đoán nhận
- Ô-tô-mát không ở trạng thái kết thúc, chuỗi không được đoán nhận

Ví dụ

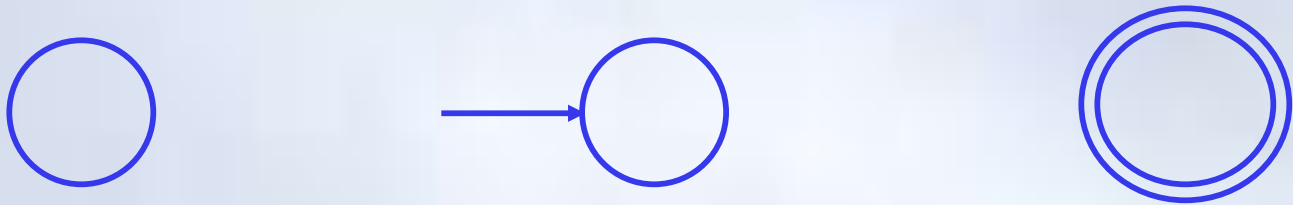
- $\Sigma = \{a, b, c\}$
- $Q = \{q_0, q_1\}$
- $q_0 = q_0$
- $F = \{q_1\}$

δ	a	b	c
q_0	q_1	q_0	q_0
q_1	q_0	q_1	q_1



Xâu *abcaabbc* được đoán nhận

Biểu diễn ô tô mát hữu hạn

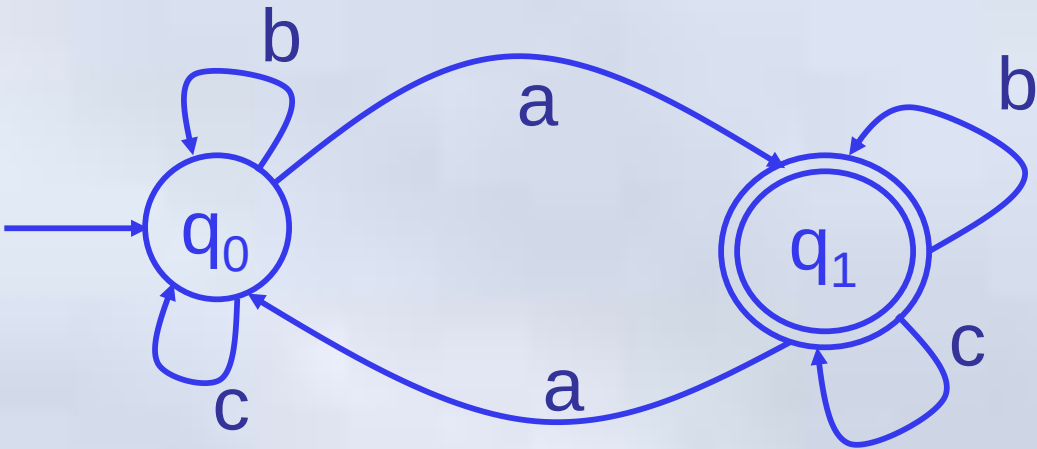
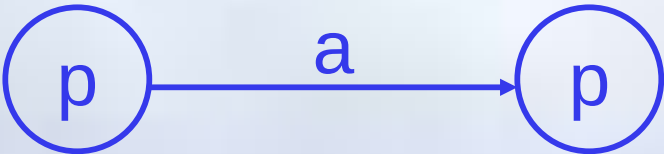


Trạng thái

Trạng thái đầu

Trạng thái kết thúc

$\delta(p,a) = q$



Xâu trên bộ chữ {a,b,c} có lẽ ký hiệu a

Ô tô máy hữu hạn đơn định (OHĐ)

OHĐ(**DFA**: Deterministic Finite Automata) là một hệ thống gồm

$$M = (\Sigma, Q, \delta, q_0, F)$$

- Σ : Bộ chữ vào
- Q : Tập hữu hạn các trạng thái của bộ điều khiển
 - $Q \cap \Sigma = \emptyset$
- $\delta : Q \times \Sigma \rightarrow Q$: Hàm dịch chuyển
 - Hàm dịch chuyển đơn định:
- $q_0 \in Q$: Trạng thái ban đầu
- $F \subseteq Q$ Tập các trạng thái cuối

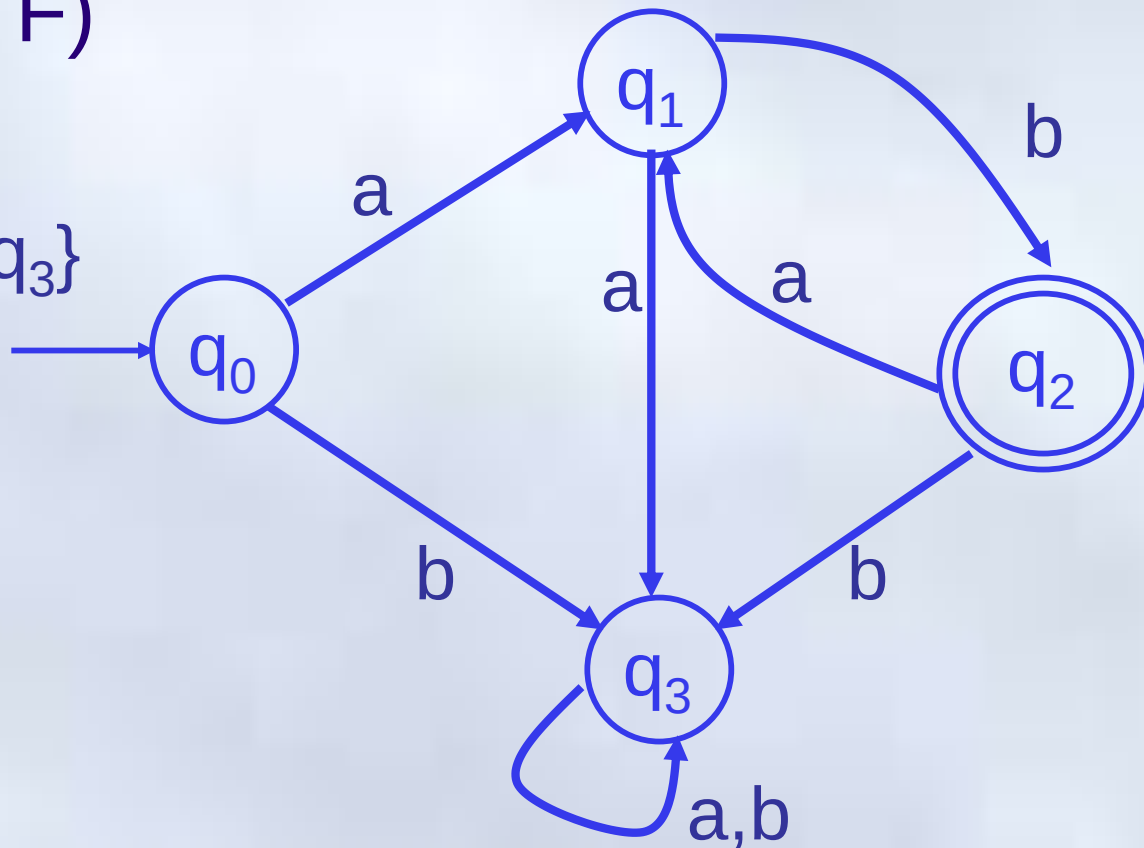
Ví dụ

$$M = (\Sigma, Q, \delta, q_0, F)$$

$$\Sigma = \{a, b\}$$

- $Q = \{q_0, q_1, q_2, q_3\}$
- $q_0 = q_0$
- $F = \{q_2\}$

δ	a	b
q_0	q_1	q_3
q_1	q_3	q_2
q_2	q_1	q_3
q_3	q_3	q_3



$$\begin{aligned}
 L(M) &= \{ab\}\{ab\}^{n \geq 0} = \{ab\}^{n(n > 0)} \\
 &= \{ab, abab, ababab, \dots\}
 \end{aligned}$$

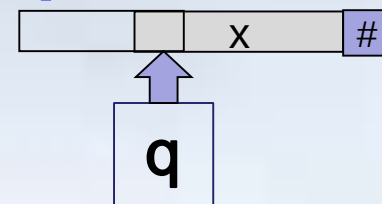
Hình trạng của FA

- Hình trạng của một FA là một chuỗi dạng **qx**

- **$q \in Q$** : Trạng thái hiện tại của FA

- **$x \in \Sigma^*$** : Phần chưa xét của chuỗi vào

- Ký hiệu được đọc bởi đầu đọc là ký hiệu đầu của x



- Chuyển đổi hình trạng

- Nếu **$x = ay$** và **$\exists \delta(q,x) = p$** thì **$qx = qay \Rightarrow py$**

$qx \Rightarrow py$: Là một bước biến đổi hình trạng

- Ví dụ: $q_0abaab \Rightarrow q_1baab \Rightarrow q_2aab \Rightarrow q_1ab \Rightarrow q_3b \Rightarrow q_3$

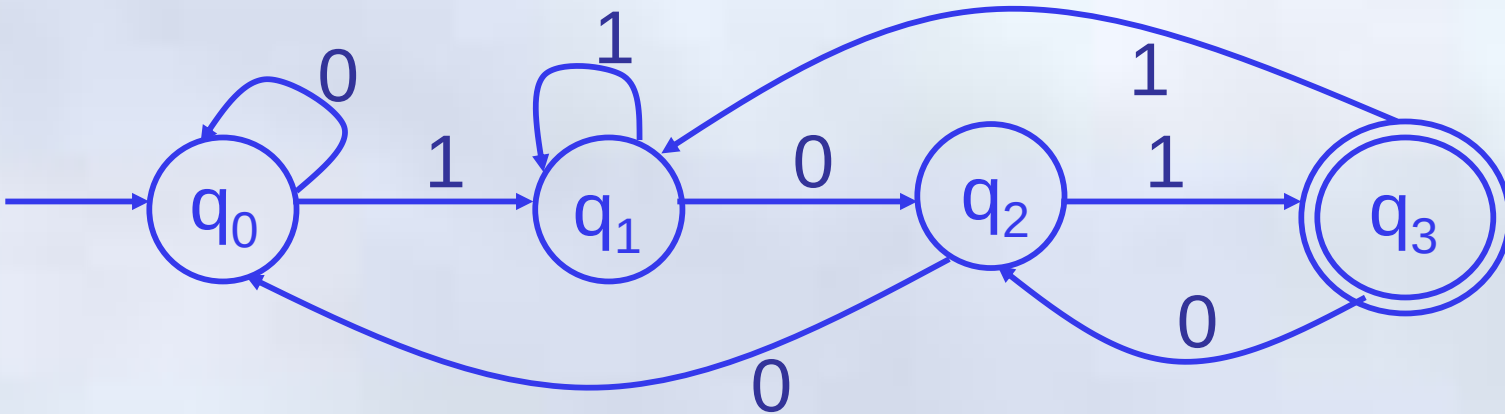
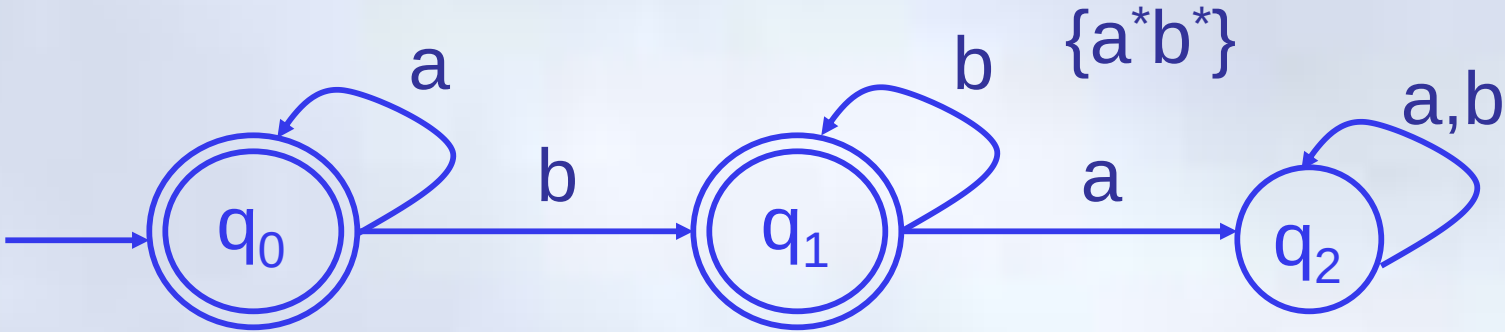
- Hình trạng đầu : **$q_0\omega$** (ω : chuỗi cần đoán nhận)

- Nếu **$q_0\omega \Rightarrow^* q_{n+1} \in F$** : Chuỗi ω được đoán nhận

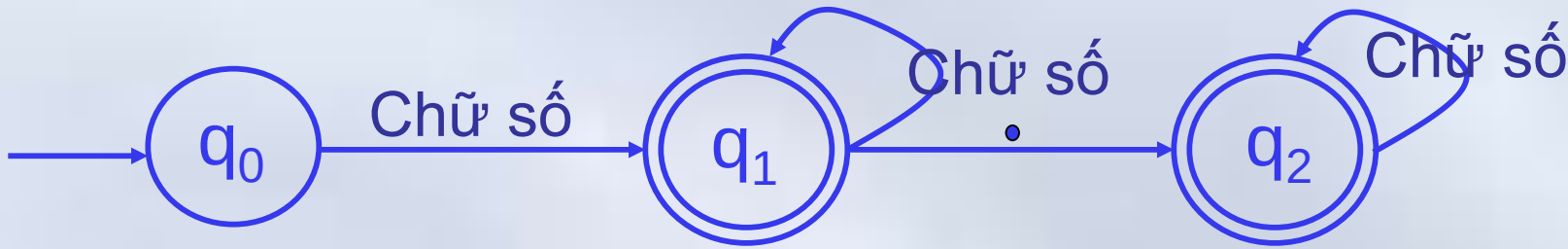
- Ngôn ngữ được đoán nhận bởi DFA M là $L(M)$

- **$L(M) = \{\omega \mid \omega \in \Sigma^* \text{ và } q_0\omega \Rightarrow^* p \in F\}$**

Ô tô mát hữu hạn đơn định→Ví dụ



$(0+1)^*101 \Leftrightarrow$ Xâu nhị phân có hậu tố là 101



Đoán nhận số nguyên hoặc số thực dấu phẩy tính

Ô tô mát hữu hạn không đơn định (OHK)

- OHK (**NFA**: Nondeterministic Finite Automata) là một hệ thống gồm

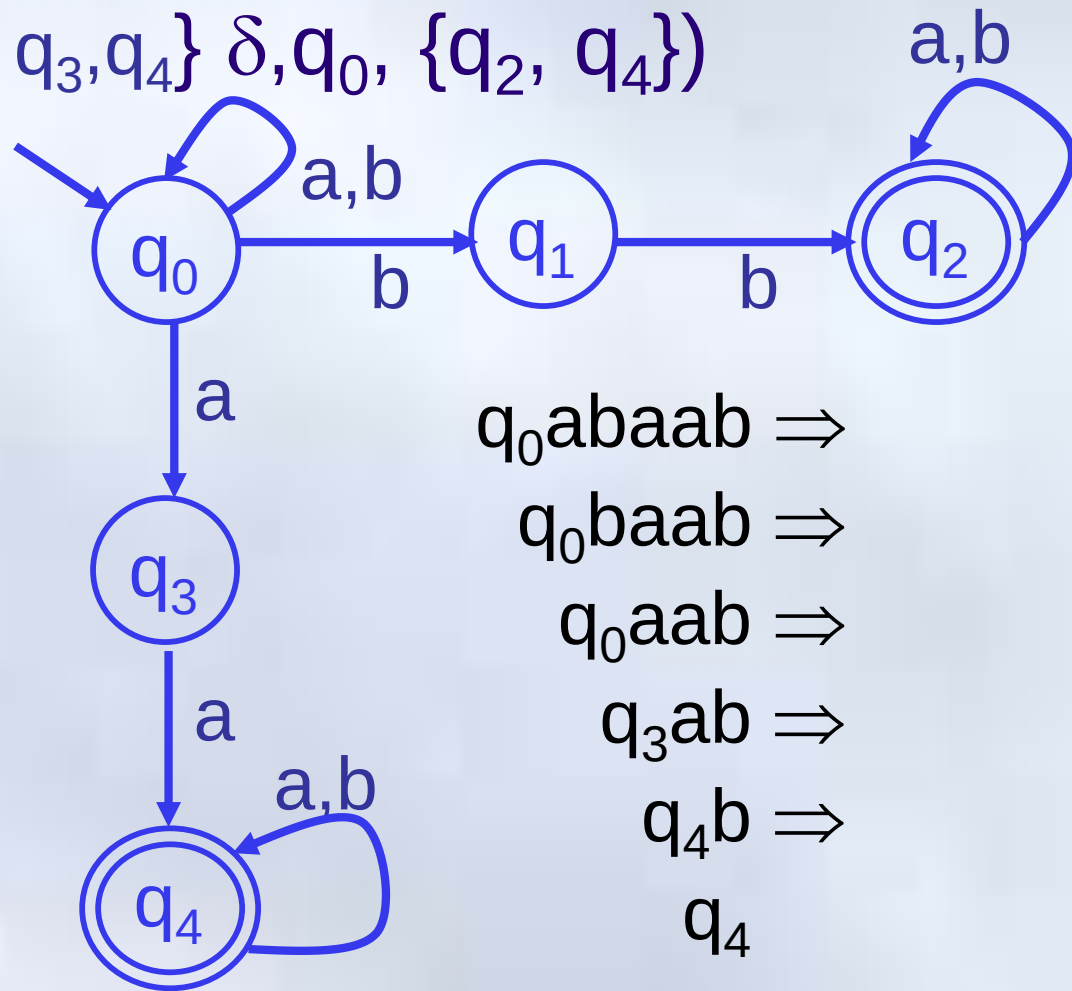
$$M = (\Sigma, Q, \delta, q_0, F)$$

- Σ, Q, q_0, F : Định nghĩa như DFA
- $\delta : Q \times (\Sigma \cup \varepsilon) \rightarrow 2^Q$
 - 2^Q : Tập các tập con của Q , kể cả tập rỗng
 - Hàm dịch chuyển không đơn định: Tại mỗi bước có thể tồn tại nhiều lựa chọn
- Đoán nhận sâu: Quá trình chuyển đổi hình trạng.
 - Quá trình đoán nhận không đơn định
- Ngôn ngữ: $L(M) = \{\omega \mid \omega \in \Sigma^* \text{ và } \exists q_0 \omega \Rightarrow^* p \in F\}$

Ví dụ

$$M = (\{a,b\}, \{q_0, q_1, q_2, q_3, q_4\}, \delta, q_0, \{q_2, q_4\})$$

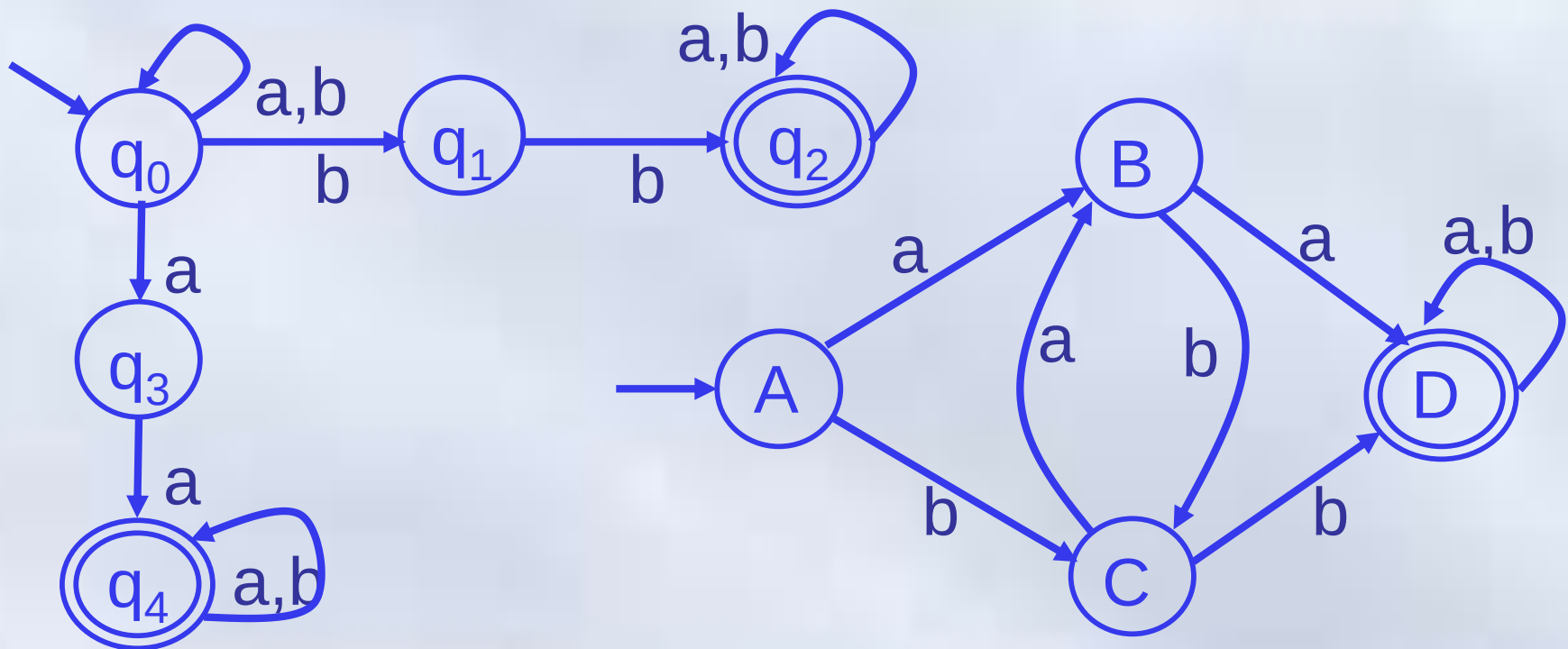
δ	a	b
q_0	$\{q_0, q_3\}$	$\{q_0, q_1\}$
q_1	$\emptyset$	$\{q_2\}$
q_2	$\{q_2\}$	$\{q_2\}$
q_3	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$



Đoán nhận xâu có 2 ký tự a, hoặc 2 ký tự b liên tiếp

DFA và NFA

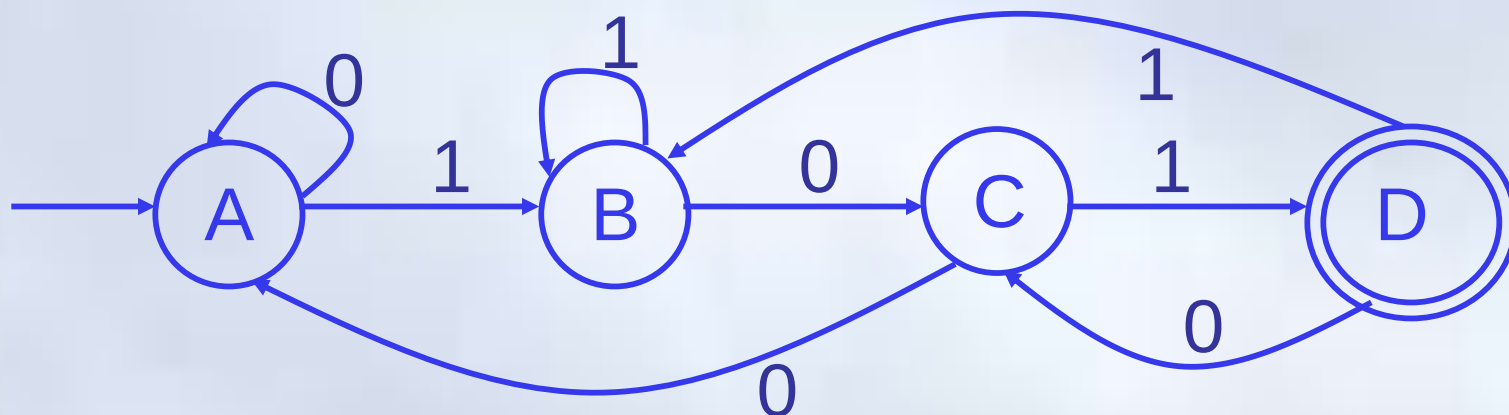
- DFA và NFA tương đương nhau
 - DFA và NFA cùng đoán nhận một lớp ngôn ngữ
 - Ngôn ngữ chính quy
- Với mỗi NFA, tồn tại DFA tương đương



Thuật toán đoán nhận xâu của OHĐ

- Phương pháp phân tích bảng
 - Dựa trên giải thuật tổng quát để đoán nhận DFA
 - Ưu điểm:
 - Chương trình độc lập với DFA
 - Dễ biến đổi, không cần sửa lại chương trình
 - Nhược điểm:
 - Khó khăn trong lập bảng
 - Kích thước bảng lớn
- Phương pháp diễn giải
 - Thực hiện như diễn giải sơ đồ
 - Dễ viết, nhưng c\trình gắn với đồ thị dịch chuyển
 - Được sử dụng để xây dựng bộ phân tích từ vựng

Phương pháp phân tích bảng



```
#include <stdio.h>
```

```
#include <string.h>
```

```
enum state {A,B,C,D};
```

//Hàm dịch chuyển dạng bảng

```
enum state Delta[4][2] = {{A,B},{C,B},{A,D},{C,B}};
```

```
char c, str[100];
```

```
int i, L;
```

```
enum state q = A;
```

//Automat ở trạng thái đầu

Phương pháp phân tích bảng

```

int main(){
    printf("Nhap xau.. :");      fflush(stdin); gets(str);
    i = 0; L = strlen(str);      c = str[i]-48;
    while (i < L){
        if(c == 0 || c == 1){    //Xâu vào chỉ gồm các ký hiệu 0,1
            q = Delta[q][c];    //Chuyển trạng thái mới
            i++;                //Dịch chuyển đầu đọc
            c = str[i]-48;      //Ký hiệu đọc được
        } else { printf("Loi xau vao...\n"); break; }
    }
    if(i==L) //Nếu đọc hết toàn bộ xâu vào
        if (q == D) printf("\n Xau %s duoc doan nhan !\n\n",str);
        else printf("\n Xau %s khong duoc doan nhan !\n\n",str);
    return 0;
}

```

Bài tập thực hành

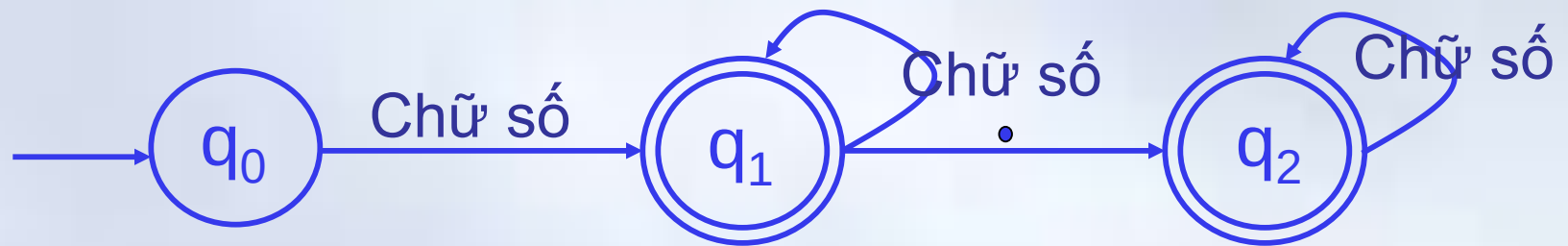
Xây dựng chương trình mô phỏng hoạt động Otomat đoán nhận ngôn ngữ biểu diễn bởi **btcq**: $(a+b)^*abb$

- **Đầu vào:** file chứa DS các xâu, các xâu phân cách bởi 1 hoặc nhiều dấu trắng.
- **Đầu ra:** các bộ: Xâu YES/NO
- Ví dụ: **Input** : ababb abbba babbbabb abba

Output

ababb	YES
abbba	NO
babbbabb	YES
abba	NO

Phương pháp diễn giải



```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main(){
    char str[100];
    int i, L;
    printf("Nhap xau.. :"); fflush(stdin); gets(str);
    i= 0; L = strlen(str);
```

Phương pháp phân tích bảng

```

if(isdigit(str[i])){           //Nếu ký hiệu đọc được là một chữ số
    i=i+1;                     //Đọc ký hiệu tiếp trên xâu vào
    while(isdigit(str[i])) i = i + 1; // đang q1:vẫn là chữ số, đọc tiếp
    if(i==L) printf("%s la so nguyen \n",str); //Đọc hết xâu vào
    else if(str[i]!='.') printf("%s khong duoc doan nhan (%d)\n",str,i+1);
    else {                     //Đã đọc được dãy số và dấu '.'
        i = i + 1;
        while(isdigit(str[i])) i = i + 1; //đang ở q2
        if(i==L) printf("%s la so thuc dau phay tinh \n",str);
        else printf("%s khong duoc doan nhan(%d)\n",str,i+1);
    }
} else printf("%s khong duoc doan nhan (%d)\n",str,i+1);
return 0;
} //main

```

Chương 2: Phân tích từ vựng

1. Nhiệm vụ của bộ phân tích từ vựng
2. Biểu thức chính quy
3. Ô tô mát hữu hạn
4. Phân tích từ vựng của ngôn ngữ PL/0

Các từ vựng của PL/0 mở rộng

- Số nguyên
- Định danh
- Từ khóa:

**begin, end, if, then, while, do, call, odd, to
const, var, procedure, program, else, for**

- Dấu phép toán:

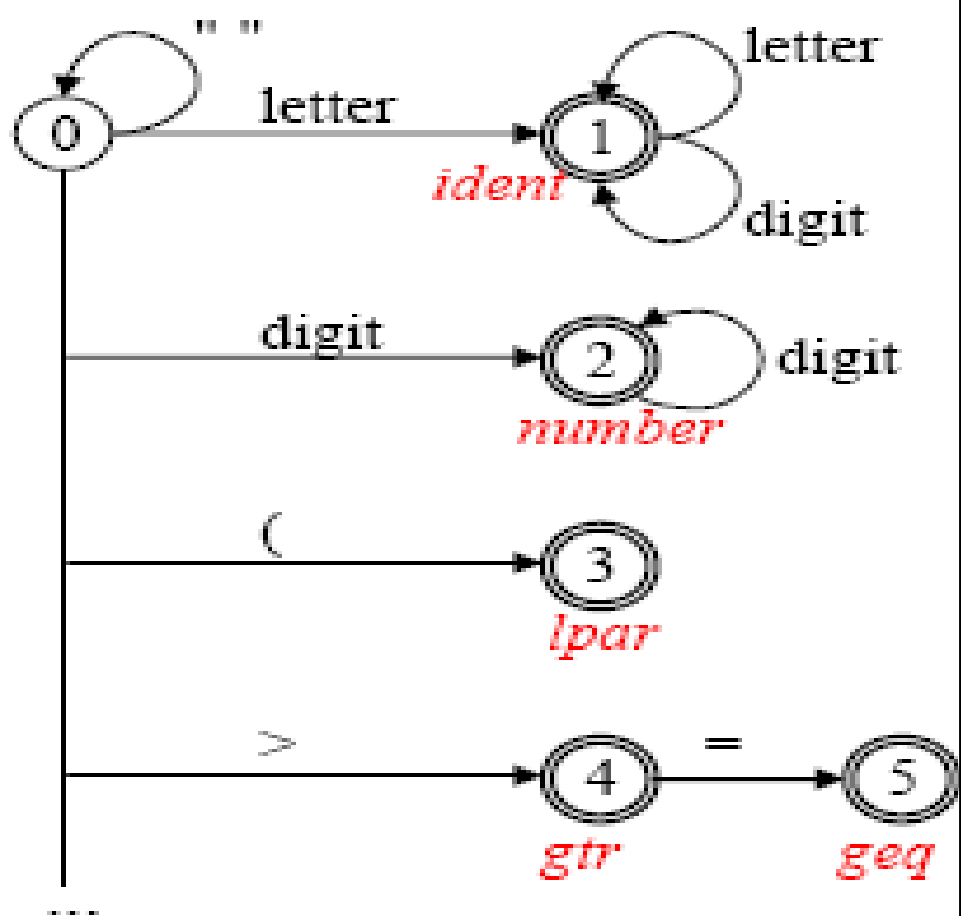
– Số học: + - * /

– So sánh: = <> < > <= >=

- Dấu phân cách: () . , ; []

- Dấu phép gán: :=

Ô-tô-mát hữu hạn của bộ phân tích từ vựng



Xâu vào: max >= 30

$s_0 \xrightarrow{m} s_1 \xrightarrow{a} s_1 \xrightarrow{x} s_1$
Ở s_1 đọc " " không chuyển trạng thái
Đoán nhận ident

$s_0 \xrightarrow{" "} s_0 \xrightarrow{>} s_4 \xrightarrow{=} s_5$
Loại bỏ khoảng trống
Không dừng ở s_4
Ở s_5 đọc " " không chuyển trạng thái
Đoán nhận geq

$s_0 \xrightarrow{" "} s_0 \xrightarrow{3} s_2 \xrightarrow{0} s_2$
Loại bỏ khoảng trống ở đầu
Ở s_2 đọc " " không chuyển trạng thái
Đoán nhận number

Sau khi mỗi từ tố được nhận biết, bộ phân tích từ vựng lại quay về trạng thái 0 $\Rightarrow$ Trạng thái bắt đầu của thủ tục đoán nhận

Các từ tổ của PL/0 mở rộng

- Mỗi từ tổ là tên một trạng thái
 - Từ tổ là **ident**, cần kiểm tra xem từ vựng tương ứng có phải là từ khóa không
- Các từ tổ của PL/0
 - Số nguyên: **NUMBER**
 - Định danh: **IDENT**
 - Nếu từ vựng trùng từ khóa, từ vựng sẽ mang ý nghĩa từ tổ trùng với tên từ khóa
 - Ví dụ: Từ vựng **Begin** có từ tổ **BEGIN**
Từ vựng **Procedure** có từ tổ **PROCEDURE**

Các từ tổ của PL/0 mở rộng

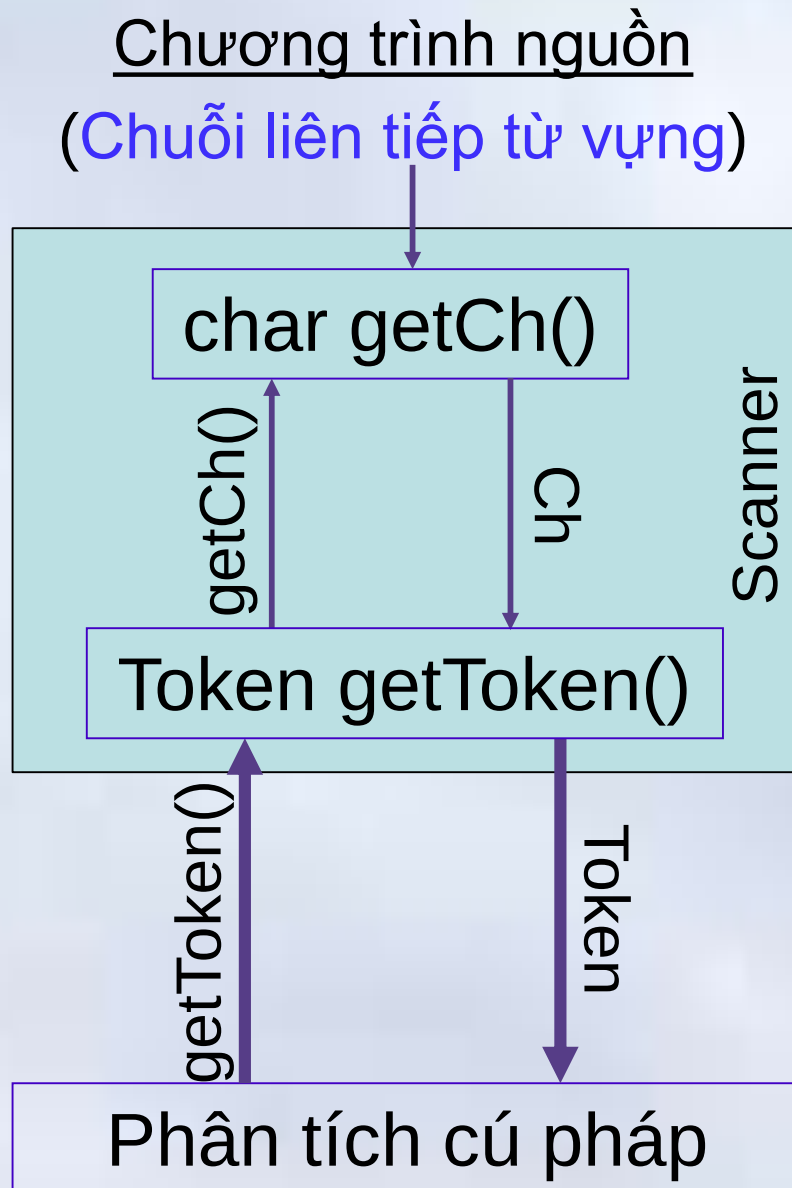
Các toán tử

Từ vựng	Từ tổ	Từ vựng	Từ tổ
+	PLUS	-	MINUS
*	TIMES	/	SLASH
=	EQU	<>	NEQ
<	LSS	<=	LEQ
>	GTR	>=	GEQ
(	LPARENT	)	RPARENT
[	LBRACK	]	RBRACK
:=	ASSIGN	;	SEMICOLON
,	COMMA	.	PERIOD
%	PERCENT		NONE

Các khai báo

```
typedef enum {//Các loại Token được sử dụng trong PL/0  
    NONE=0, IDENT, NUMBER,  
    BEGIN, CALL, CONST, DO, ELSE, END, FOR, IF, ODD,  
    PROCEDURE, PROGRAM, THEN, TO, VAR, WHILE,  
    PLUS, MINUS, TIMES, SLASH, EQU, NEQ, LSS, LEQ,  
    GTR, GEQ, LPARENT, RPARENT, LBRACK, RBRACK,  
    PERIOD, COMMA, SEMICOLON, ASSIGN, PERCENT  
} TokenType;  
  
TokenType Token; //Token nhận dạng được  
  
int    Num;      //Từ vựng khi Token là NUMBER  
  
char  Id[MAX_IDENT_LEN + 1]; //Từ vựng khi Token là IDENT
```

Xây dựng từ vựng



- Scanner gồm 2 hàm

- `getCh()`:

- Làm việc với thiết bị lưu trữ
 - Đọc file nguồn, trả về các ký tự ASCII

- `getToken()`

- Được gọi từ Parser
 - Xây dựng từ vựng từ các ký tự trả về bởi `getCh()`
 - Nhận dạng từ tổ

- Hoạt động

- Tương tự OĐĐ

- Khi được gọi (*bắt đầu hoạt động*) ở trạng thái q_0

Xây dựng từ vựng

- Khi bộ pttv - *thủ tục getToken()* bắt đầu hoạt động, ô-tô-mát ở trạng thái khởi tạo (*Trạng thái 0*)
- Bộ pttv gọi liên tiếp *getCh()* để đọc các ký hiệu trên văn bản nguồn cho tới khi gặp một ký tự không thuộc luật mô tả hiện tại → Ô-to-mát không chuyển trạng thái được. Khi đó
 - Xâu đọc được là từ vựng mang ý nghĩa của từ tổ đang phân tích và là trạng thái hiện tại của Ô-tô-mát
 - Đọc thừa ra một ký tự
 - Là ký tự trắng hoặc ký tự đầu của từ tổ tiếp → Khi *getToken()* được gọi, sẽ làm việc ngay với một ký tự có sẵn.
 - Tại lần gọi *getToken()* đầu tiên để xác định từ tổ thứ nhất, bộ ptcp phải cũng cấp sẵn một ký tự → là ký tự trắng

Xây dựng từ vựng

```
//Ch chứa ký tự đọc được từ văn bản nguồn bởi hàm getCh()
switch(Ch)  {
    case SPACE : while (Ch==SPACE) getCh();
    case LETTER: getCh();
        while (Ch==LETTER || Ch==DIGIT) getCh();
        return Ident; //Cũng có thể là từ khóa → Phải kiểm tra
    case DIGIT: while (Ch==DIGIT) getCh(); return NUMBER;
    case '+': getCh(); return plus;
    case '>' : getCh();
        if(Ch == '=') { getCh(); return GEQ;}
        else return GTR;

    .....
}
```

Nhận dạng từ tổ

Xây dựng xong từ vựng, cần nhận dạng từ tổ

- Là tên trạng thái cuối cùng của Ô-tô-mát
- Nếu Ô-tô-mát kết thúc ở trạng thái IDENT, cần kiểm tra đây có phải là từ khóa
 - Dùng kỹ thuật bảng chuyển đổi

Chú ý: Với từ tổ ident và number, cần phải ghi nhận giá trị từ vựng tương ứng

BEGIN	→	BEGIN
CALL	→	CALL
.....		
VAR	→	VAR
WHILE	→	WHILE

Bài tập

1. Xây dựng hoàn chỉnh bộ phân tích từ vựng cho PL/0
2. Tìm hiểu về Lex/JLex